

ESERCIZI SU PATTERN MATCHING

Martedì 21 Aprile 2026



STRING INIZIALIZZAZIONE

In Java, si usano oggetti di tipo String.

Esempi:

```
String s1 = "world";  
String s2 = new String("hello");  
String s3 = "aaa".repeat(1); //s3 is "aaa"  
String s4 = ""; ///empty string  
String s5 = new String(s2); //copies s2 into s5
```

Operazioni comuni:

- s.length()
- s.isEmpty()
- s.charAt(index)
- s.substring(start, end)
- s.equals(anotherString)
- s.compareTo(anotherString)



NOTE

- strings posso essere arbitrariamente lunghe
- Quando una stringa va out-of-scope la sua memoria è liberata
- `s = "wow" // assign literal to string`
- `s1 = s2 // copy contents of s2 into s1`
- `s1 + s2 // return new string: s1 concatenated with s2`
- `s1 += s2 // same as s1 = s1 + s2, also same as s1.append(s2)`
- `== != < > <= >= // relational operators; alphabetical order`



Input/output String

- Input:
- `Scanner sc = new Scanner(System.in);`
- `String s = sc.nextLine(); // legge tutta la linea`
- `String parola = sc.next(); // legge una parola`
- Output:
- `System.out.println(s);`



Arraylist in java

- `import java.util.ArrayList;`
- `ArrayList<Integer> scores = new ArrayList<>();`
- `ArrayList<String> names = new ArrayList<>();`
- `names.add("Luca Rossi");`
- `names.add("Primo Cognome1");`
- `names.add("Secondo Cognome2");`
- `System.out.println("First name: " + names.get(0));`
- `System.out.println("Last name: " + names.get(names.size()-1));`

DUE MODI PER STAMPARE TUTTI GLI ELEMENTI

- Con un indice:

```
for (int i = 0; i < names.size(); i++) {  
    System.out.println(names.get(i));  
}
```

- Con un *iterator*:

```
// oppure con for-each  
for (String name : names) {  
    System.out.println(name);  
}
```

HashMap in Java (equivalente di **MAP**)

- `import java.util.HashMap;`
- `HashMap<Integer, String> rubrica = new HashMap<>();`
- `rubrica.put(90134, "Mariella Bonomo");`
- `System.out.println("Key=90134, Value=" + rubrica.get(90134));`
- `if (rubrica.containsKey(92394)) {`
- `System.out.println("Found it");`
- `} else {`
- `System.out.println("Didn't find it");`
- `}`

Iterare un Hasmap

```
for (Map.Entry<Integer, String> entry : rubrica.entrySet()) {  
    System.out.println(entry.getKey() + ": " +  
entry.getValue());  
}
```


Algoritmo KMP in Java - costruzione DFA

```
int[] computeLPSArray(String pat) {  
    int M = pat.length();  
    int[] lps = new int[M];  
    int len = 0;  
    int i = 1;  
  
    while (i < M) {  
        if (pat.charAt(i) == pat.charAt(len)) {  
            len++;  
            lps[i] = len;  
            i++;  
        } else {  
            if (len != 0) {  
                len = lps[len-1];  
            } else {  
                lps[i] = 0;  
                i++;  
            }  
        }  
    }  
    return lps;  
}
```

KMP Search

```
void KMPSearch(String pat, String txt) {  
    int M = pat.length();  
    int N = txt.length();  
    int[] lps = computeLPSArray(pat);  
  
    int i = 0; // index for txt  
    int j = 0; // index for pat  
    while (i < N) {  
        if (pat.charAt(j) == txt.charAt(i)) {  
            i++;  
            j++;  
        }  
  
        if (j == M) {  
            System.out.println("Found pattern at index " + (i-j));  
            j = lps[j-1];  
        } else if (i < N && pat.charAt(j) != txt.charAt(i)) {  
            if (j != 0)  
                j = lps[j-1];  
            else  
                i = i+1;  
        }  
    }  
}
```



Rotazione ciclica di una stringa

```
boolean isRotation(String s1, String s2) {  
    if (s1.length() != s2.length()) return false;  
    return (s1 + s1).contains(s2);  
}
```



Tandem Repeat

```
int maxTandemRepeat(String b, String s) {  
    int maxRepeat = 0;  
    int blen = b.length();  
    for (int i = 0; i <= s.length() - blen; i++) {  
        int count = 0;  
        while (s.startsWith(b, i + count * blen)) {  
            count++;  
        }  
        maxRepeat = Math.max(maxRepeat, count);  
    }  
    return maxRepeat;  
}
```



Calcolare il periodo di una stringa

```
int findPeriod(String s) {  
    int[] lps = computeLPSArray(s);  
    int len = s.length();  
    int p = len - lps[len-1];  
    if (len % p == 0) return p;  
    else return len;  
}
```



KMP2: ricerca multipla

```
List<Integer> KMP2(List<String> patterns, String text) {  
    List<Integer> result = new ArrayList<>();  
    for (String pattern : patterns) {  
        int index = text.indexOf(pattern);  
        result.add(index);  
    }  
    return result;  
}
```



ESERCIZI

- Utilizzando le strutture dati precedentemente descritte, implementare l'algoritmo KMP e l'algoritmo KMP2 discussi nella lezione precedente



ESERCIZIO

- Costruire il DFA dell'algoritmo KMP per i seguenti pattern:
 - AAAAAB
 - AAAAAA
 - AACAAAB
 - ABABABAB
 - ABAABAAABAAAB
 - ABAABCABAABCB



ESERCIZI

1. Implementare l'algoritmo KMP che, dato un pattern e un testo, trovi tutte le occorrenze del pattern nel testo.
2. Implementare un algoritmo lineare per determinare se una data stringa è rotazione ciclica di un'altra
3. Un **tandem repeat** di una stringa base b all'interno di una stringa s è una sottostringa di s che consiste di almeno una copia consecutiva della stringa base b . Dati b e s , progettare un algoritmo per trovare una tandem repeat di b di lunghezza massima all'interno di s . Il tempo di esecuzione dovrebbe essere proporzionale a $M + N$, dove M è la lunghezza di b e N è la lunghezza di s . Per esempio, il programma dovrebbe restituire 3 se b è **abcb** e s è **abcabcbabcbabcbabc**
4. Sia s una stringa non vuota. Un intero positivo p tale che $s[i]=s[i+p]$ per ogni $i=0,1,\dots,N-p-1$, è chiamato un periodo di s . Il **periodo** della stringa s è il più piccolo di tali interi. Per esempio il periodo di ABCABCABCABC è 3. Progettare un algoritmo lineare che calcoli il periodo di una stringa.
5. Sia s una stringa non vuota. La stringa w è detta un bordo di s se $s=yw=wz$ con $|y|=|z|=p$, cioè w è un suffisso proprio che è anche prefisso. Il **bordo** di una stringa s è il più lungo dei suoi bordi propri (può essere vuoto). Per esempio, il bordo di ABCABCABCABC è $w=ABCABCABC$ (con $y=ABC$, $z=CAB$, $p=3$). Progettare un algoritmo lineare che calcoli il bordo di una stringa.
6. Implementare l'algoritmo KMP2 che data una lista di pattern e un testo, restituisca una lista con le prime occorrenze di ciascun pattern nel testo. Qual è la complessità di tempo?
7. Gestire il testo come input da file.

